Draft version July 23, 2026

Optimizing WHFast512 in x86 Assembly: A Faster and More Accurate Symplectic Integrator for Planetary Systems

Rishit Dagli[1] and Hanno Rein[1, 2, 3, 4]

[1] Dept. of Computer Science, University of Toronto, Toronto, 40 St. George Street, Toronto, ON M5S 2E4, Canada
[2] Dept. of Physical and Environmental Sciences, University of Toronto at Scarborough, Toronto, Ontario, M1C 1A4, Canada
[3] Dept. of Physics, University of Toronto, Toronto, Ontario, M5S 3H4, Canada
[4] Dept. of Astronomy and Astrophysics, University of Toronto, Toronto, Ontario, M5S 3H4, Canada

ABSTRACT

Longterm integrations of planetary systems are used in a variety of astrophysical applications, including determining a system's stability. In this paper, we describe significant speed and accuracy improvements to the `WHFast512` integrator. By completely rewriting `WHFast512` in x86 assembly, we gain fine-grained control over CPU instructions and are able to keep the entire simulation state in CPU registers, only writing to memory when an output is required. We also reduce the number of branches to two per timestep, improve the Kepler solver's speed and accuracy with a convergence check, switch the Hamiltonian splitting to Jacobi coordinates, implement symplectic correctors, and add support for ejection and collision detection. With these improvements, we can integrate all eight planets of the Solar System with general relativistic corrections for 5 Gyr in under 8.7 hours, 4× faster than the runtime reported in the original `WHFast512` paper and almost 10× faster than integrators that do not use AVX512 instructions. This makes `WHFast512` by far the fastest N-body integrator for this kind of simulation. We run extensive tests to verify the integrator's accuracy, with a particular focus on removing any long-term bias, and show that the accuracy is now five orders of magnitude better for typical setups. The new `WHFast512` integrator is freely available in the `REBOUND` integrator package.

Keywords: N-body simulations (1083) — Celestial mechanics (211) — Computational methods (1965) — Solar system evolution (2293) — Planetary system evolution (2292)

1. INTRODUCTION

Symplectic integrators that make use of the Wisdom-Holman splitting (J. L. Wisdom 1981; J. Wisdom & M. Holman 1991; H. Kinoshita et al. 1991) are widely used to integrate particles in systems where the motion is dominated by one central object such as planetary systems. Many different variants of the Wisdom-Holman (WH) integrator exist including several high order methods. For an overview, see e.g. H. Rein et al. (2019).

In contrast to many other problems in computational astrophysics, N-body simulations with small N, such as simulations of planetary systems, cannot easily be parallelized because they are inherently sequential. Timesteps are performed one after another because the result of each timestep is required for the next one[5]. As such, even on modern CPUs and with access to large computer clusters, an individual N-body simulation of the Solar System can take days or weeks to run.

Over the years, improvements in computer performance, particularly CPU speed, have significantly reduced the runtime of astrophysical simulations. In Fig. 1 we plot the wall-clock time (elapsed real time) required to integrate the Solar System with all eight planets for 5 Gyr, as reported in 13 publications spanning eight decades. The wall-clock time has been normalized to account for different timesteps and numbers of planets. As indicated by the dashed line, the wall-clock time falls off exponentially, halving on average every 2.5 yr. This is consistent with Moore's law (G. E. Moore 1965), which predicts that the number of transistors on a computer chip doubles approximately every 2 yr (G. E. Moore 1975).

What Fig. 1 hides is that CPU clock rates have barely increased since approximately the year 2000 (A. Danowitz et al. 2012). This is due to physical constraints, such as CPU heat dissipation, and architectural choices that favor parallel processing of large data sets,

[5] P. Saha et al. (1997) use a time parallel method with some success in terms of runtime, but it comes with a high computational cost.

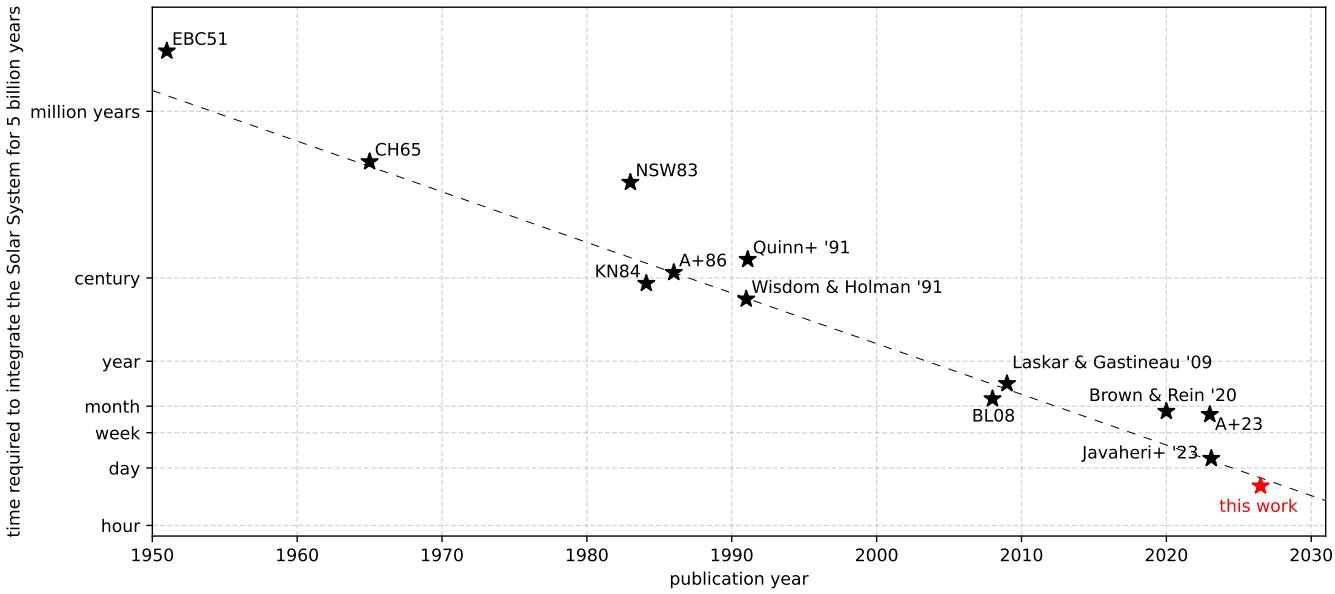

Figure 1. The wall-clock time required for an integration of the Solar System with all eight planets for 5 Gyr as reported by W. J. Eckert et al. (1951); C. J. Cohen & E. C. Hubbard (1965); X. X. Newhall et al. (1983); H. Kinoshita & H. Nakai (1984); J. H. Applegate et al. (1986); T. R. Quinn et al. (1991); J. Wisdom & M. Holman (1991); K. Batygin & G. Laughlin (2008); J. Laskar & M. Gastineau (2009); D. S. Abbot et al. (2023) and P. Javaheri et al. (2023). The bottom right data point shows the result reported in this paper. The four rightmost data points all use a version of either `WHFast` or `WHFast512`. The dashed line shows an exponential function, halving the walltime every 2.5 yrs. The data has been adopted from Sam Hadden, https://shadden.github.io/nbody_history/.

including multi-core CPUs and GPUs (H. Esmaeilzadeh et al. 2011). The performance gains shown in Fig. 1 over the last two decades, particularly those reported by P. Javaheri et al. (2023) and the present work, are not due to higher clock rates but rather due to improvements in algorithms and the use of advanced CPU instructions.

AVX512 describes a set of CPU features which extends the standard x86 instruction set by adding new instructions that operate on 512-bit wide registers. Such a register can store up to 8 double precision floating point numbers. One AVX512 instruction can then operate on all 8 floating point numbers in parallel. This makes it ideal for scientific computing where many floating point operations are required. Parallelization with AVX512 has the big advantage that all operations happen on one CPU, thus avoiding the need for communication between different CPU/GPU cores or cluster nodes. This is particularly important for the kind of astrophysical simulation we are concerned here as one timestep can take as little as 100ns[6]. Some compilers can automatically generate AVX512 instructions, but as we show in this paper, their capabilities are nowhere near what can be achieve by assembling the instructions by hand. The fact that the Solar System has 8 planets is a fortunate coincidence as it allows us to fully occupy a register with exactly one floating point number for each of the 8 planets.

Our work builds on that described in P. Javaheri et al. (2023), which uses explicit AVX512 instructions for N-body simulations of planetary systems. This first version of the `WHFast512` integrator was written in C99 using intrinsic functions for the AVX512 instructions. Although this approach is convenient from a programming perspective, it leaves many decisions about register use and the order of operations to the compiler. The present paper describes several significant improvements to the `WHFast512` integrator, which we refer to as `WHFast512` Version 2. Specifically, we reduce the wall-clock time by a factor of 4 while increasing the accuracy by 5 orders of magnitude.

We note that K. Nitadori et al. (2006) and A. Tanikawa et al. (2012) also use SSE and AVX instructions to speed up N-body simulations. Their methods use a Hermite scheme and target star clusters with more particles than the systems considered here, $N \sim 10^3 - 10^5$. Like us, they accelerate their algorithms by writing crucial parts in assembly.

The plan for this paper is as follows. In Section 2 we describe the algorithmic improvements that we made such as switching to Jacobi coordinates and changes to

[6] This corresponds to a light travel time of only 30m!

the Kepler solver and the Stiefel functions. We then focus on the performance optimizations in Section 3 where we discuss our choice to reimplement `WHFast512` in assembly, the advantages of keeping the entire simulation in registers, an efficient matrix multiplication algorithm, how we avoid branching, and when the use of approximate operations is acceptable. Various tests are presented in Section 4 which demonstrate the accuracy and performance of our new integrator. We discuss in Section 5 why we think our integrator is almost optimal, making it very hard to achieve further speed improvements. We conclude in Section 6.

2. ALGORITHMIC IMPROVEMENTS

The main idea of a WH integrator is to split the motion of each planet into two parts: motion around the central object on a Keplerian orbit and perturbations from interactions with the other planets. This perturbative approach reduces the error, relative to a non-perturbative integrator, by a factor comparable to the Jupiter-to-Sun mass ratio, roughly 10^{-3} in the Solar System.

To perform this splitting, a WH integrator must solve two problems in a leap-frog fashion. First, planets are moved along Keplerian orbits in the *Kepler step*. Kepler's equation has no closed form solution, so this step is performed iteratively. Second, forces from planet-planet interactions are applied in the *interaction step*. This step can be computed explicitly in any inertial frame but can be computationally expensive because it involves $N\cdot(N-1)/2$ force evaluations.

Parallelizing the Kepler and interaction steps poses different challenges and we describe the algorithmic improvements in this section. For a more comprehensive introduction to symplectic integrators and specifically the `WHFast` and `WHFast512` integrators, see H. Rein & D. Tamayo (2015) and P. Javaheri et al. (2023).

2.1. *Jacobi Coordinates*

The separation into Keplerian motion and a perturbation part is not unique and can be done in different ways, each of which has its own advantages and disadvantages (see H. Rein & D. Tamayo 2019 for an overview of the most commonly used Hamiltonian splittings). The first version of `WHFast512` used democratic heliocentric coordinates (DHCs) for the Hamiltonian splitting. This does not require coordinate transformations between the interaction and Kepler steps. Furthermore, planets do not need to be ordered. However, for the same timestep, a Hamiltonian splitting based on DHCs is less accurate than one based on Jacobi coordinates (A. Farrés et al. 2013). In particular, DHCs are not well suited for long-term integrations of planetary systems, as described in detail in H. Rein et al. (2026). This is because the DHC splitting of the Hamiltonian introduces an additional numerical precession that can easily dominate over physical effects. To avoid unphysical results, one needs to reduce the timestep by almost an order of magnitude. However, doing so eliminates the potential speed-up gained by `WHFast512`.

For this reason, we use Jacobi coordinates in the new version of `WHFast512`. The main downside of Jacobi coordinates is that they require coordinate transformations between each Kepler step and interaction step. Specifically, one needs to transform the positions from Jacobi coordinates to heliocentric coordinates before the interaction step and transform the accelerations from heliocentric coordinates to Jacobi coordinates afterward.

One key insight that allows us to implement these coordinate transformations efficiently is that they can be written as linear transformations, specifically 8×8 triangular matrices for an eight-planet system. The matrix coefficients are constants and can be calculated once at the beginning of the simulation because they depend only on the particle masses, not on their positions or velocities (see, e.g., H. Beust 2003). Our specific implementation of an efficient matrix-vector multiplication is described in Section 3.3.

Another advantage of a Hamiltonian splitting in Jacobi coordinates is that it allows for symplectic correctors, which can reduce errors by orders of magnitude with almost zero overhead (J. Wisdom et al. 1996). We implement symplectic correctors of order 17 using a concatenation of Kepler and interaction steps.

2.2. *General Relativity, Close Encounters, and Ejections*

The stability of planetary systems, particularly the Solar System, is sensitive to the precise precession frequencies of the planets (J. Laskar 2008; J. Laskar & M. Gastineau 2009; G. Boué et al. 2012; G. Brown & H. Rein 2023). We therefore implement an option to include a general relativistic correction to Newtonian gravity in the form of an additional $1/r^2$ potential centered on the central object. This first order correction mimics the effects of general relativistic precession (A. Nobili & I. W. Roxburgh 1986). Because we use Jacobi coordinates, we already calculate the heliocentric distance of each planet in the interaction step. Thus, adding this correction term has minimal cost because no additional square roots or divisions are required.

We also implement optional checks for close encounters and ejections at every timestep. The ejection check compares the heliocentric distance of each planet with

a preset value. The simulation stops when the distance exceeds this value. Similarly, the encounter check compares the distance between each pair of planets with a preset value. The simulation stops when the distance falls below this value. Once again, we can reuse distances that are already calculated in the interaction step, so these checks do not add significant overhead.

All three optional features increase the runtime by at most a few percent.

2.3. *Kepler Solver*

Since no closed form solution to Kepler's equation exists, this non-linear problem must be solved iteratively. In the first version of `WHFast512`, an initial guess was refined using a fixed number of iterations. This works for packed systems such as the Solar System because we can assume a priori that the planets occupy only a restricted range of orbital parameters (eccentricity and orbital period), as they would otherwise collide with one another. However, this approach fails for other systems, particularly those in which high eccentricities might occur temporarily without leading to an instability. For this reason, we improve the Kepler solver in this version of `WHFast512`.

We follow the notation of S. Mikkola (1997) and H. Rein & D. Tamayo (2015) and write Kepler's equation as

$$r_0 X + \eta_0 Gs_2(\beta, X) + \zeta_0 Gs_3(\beta, X) - dt = 0, \quad (1)$$

where Gs_n are Stiefel functions (E. L. Stiefel & G. Scheifele 1975), which in turn depend on Stumpff functions c_n that we implement using a truncated Taylor series (see Eq. 2). The goal is to find X given the timestep dt and the quantities $r_0, \beta, \eta_0, \zeta_0$, all of which depend on the positions and velocities at the beginning of the timestep.

Our new algorithm works as follows. We start with a first order guess for $X = dt/r_0$. We then perform two Halley steps to refine X. In the first step, we include only 9 terms in the Taylor expansion of the Stumpff functions. In the second step, we include 11 terms. Now that the estimate is relatively close to the solution, we refine it with a Newton step for which we include 19 terms in the Stumpff functions. At this point we introduce a convergence check, similar to what is present in `WHFast` but is absent in the first version of `WHFast512`. We refine the solution for each planet with Newton iterations until it has converged according to an empirically determined criterion $|\delta X| < \epsilon = 10^{-11}$. We found that if we have not yet converged after four Netwon iterations, then we are likely to not converge at all. Thus, if a converged solution is not obtained after the first four Newton iterations, then we switch to a fallback bisection method. Once the solution has converged, we calculate the f and g functions used to update the positions and velocities.

With AVX512, we can perform the Kepler step in parallel for eight planets. However, the slowest converging solution (typically that of the innermost or most eccentric planet) determines the maximum number of iterations for all planets. To ensure that each planet's solution remains independent of all other planets in the simulation, only the lanes for planets that have not yet converged are updated at each step. This is important because an excessive number of iterations on an already converged solution can actually worsen the accuracy due to accumulating roundoff errors. We achieve this using masked instructions that write only to the lanes corresponding to planets that have not yet converged. Masked instructions themselves have no overhead in terms of latency and throughput, but we require some CPU cycles to generate and update the masks at each iteration.

The specific sequence of Halley and Newton steps and the number of terms used in the Taylor series expansion of the Stumpff functions were identified through a brute-force comparison of all possible combinations, subject to the requirement of convergence to machine precision. We then chose the most efficient sequence for a typical Solar System integration. For different applications, for example in cases with highly eccentric orbits, a different sequence might be slightly faster.

To summarize our implementation of the Kepler solver, we list the pseudocode in Algorithm 1. The Kepler solver is evaluated for all 8 planets in parallel using AVX512 instructions. Note that for readability, we do not show the masked instructions or the mask generation.

2.4. *Roundoff in the Stiefel functions*

We need to evaluate Stiefel functions Gs for the Kepler solver described above. The Stiefel functions in turn depend on the Stumpff functions c_n. We calculate the Stumpff functions from a truncated Taylor series:

$$c_n(z) \equiv \sum_{j=0}^{n_{\text{terms}}} \frac{(-z)^j}{(n+2j)!}. \quad (2)$$

In floating point arithmetic, this evaluation carries a small but systematic bias. The bias has two different origins. First, the inverse factorial coefficients of the series are pre-calculated and rounded to the nearest representable number. Second, the summation of the series involves rounding at every term. Both effects are tiny,

Input: dt, positions, velocities
Output: positions, velocities
$r, \beta, \eta, \zeta \leftarrow$ from positions and velocities
`/* First order guess */`
$X \leftarrow dt/r$
`/* First two refinements */`
$Gs_{0,1,2,3} \leftarrow$ Stiefel(X, β, nTerms $= 9$)
$X \leftarrow$ Halley(X, dt, $Gs_{0,1,2,3}, \beta, \eta$)
$Gs_{0,1,2,3} \leftarrow$ Stiefel(X, β, nTerms $= 11$)
$X \leftarrow$ Halley(X, dt, $Gs_{0,1,2,3}, \beta, \eta$)
`/* Newton iteration loop */`
$X' \leftarrow X$
for $i \leftarrow 0$ **to** 4 **do**
  $Gs_{1,2,3} \leftarrow$ Stiefel(X, β, nTerms $= 19$)
  $X \leftarrow$ Newton(X, dt, $Gs_{1,2,3}, \beta, \eta$)
  if $|X - X'| < \epsilon$ **then**
    goto converged
  $X' \leftarrow X$
`/* Fallback bisection method */`
Period $\leftarrow$ from positions and velocities
$X_{min} \leftarrow 2\pi \text{ floor}(dt/\text{Period})/\sqrt{\beta}$
$X_{max} \leftarrow X_{min} + 2\pi/\sqrt{\beta}$
$X \leftarrow (X_{min} + X_{max})/2$
for $i \leftarrow 0$ **to** 51 **do**
  $Gs_{1,2,3} \leftarrow$ Stiefel(X, β, nTerms $= 19$)
  if $r\,X + \eta\,Gs_2 + \zeta\,Gs_3 < dt$ **then**
    $X_{min} \leftarrow X$
  else
    $X_{max} \leftarrow X$
  $X \leftarrow (X_{min} + X_{max})/2$
`/* Final update */`
converged:
$Gs_{1,2,3} \leftarrow$ StiefelCompensated(X, β, nTerms $= 19$)
$f, g, \dot{f}, \dot{g} \leftarrow$ using $Gs_{1,2,3}, \eta, \zeta, dt$
new positions $\leftarrow$ using f, g, positions, velocities
velocities $\leftarrow$ using $\dot{f}, \dot{g}$, positions, velocities
positions $\leftarrow$ new positions

Algorithm 1. Pseudocode of the new `WHFast512` Kepler solver. $Gs_{0,1,2,3}$ are the Stiefel functions defined in S. Mikkola (1997) and implemented using Algorithm 2. The quantities X, β, η, and ζ are defined in H. Rein & D. Tamayo (2015).

of the order of 10^{-20} per timestep, but they can accumulate over time because the rounding errors in the coefficients are constant. This causes the energy error in long integrations to grow linearly in time.

This bias is present in the first version of `WHFast512`, although it is hardly visible in a typical simulation because the overall scheme error is significantly larger than in our new version (see Section 2.1). Here, we aim to implement a less biased scheme in which the energy error grows only with the square root of the number of steps, consistent with Brouwer's law (D. Brouwer 1937).

We reduce the bias with the following two changes. First, we precalculate the rounding errors of the inverse factorial coefficients of the series in extended precision. Second, we evaluate the series with a compensated summation that includes both the original inverse factorial coefficients and rounding error corrections. The compensated summation retains the low order bits that would otherwise be lost. Neither change is sufficient on its own because the coefficient corrections are too small to be represented in double precision and are simply discarded unless the compensated summation also carries the extra bits. Only the last few terms of the series require this correction because, for a typical timestep, the correction is small and the higher order terms are negligible. Furthermore, only the final evaluation of c_n that we ultimately use for the f and g functions requires this correction because this evaluation advances the particles' positions and velocities. Intermediate evaluations used inside the iterations of the Kepler solver do not significantly affect the final state. The correction therefore adds only a handful of operations to the series evaluation per timestep, but it restores the expected random walk behavior of the energy error over multi-Gyr integrations, as we show in Section 4.3.

We list the pseudocode for our Stiefel function evaluation in Algorithm 2. The Stiefel functions are evaluated for all 8 planets in parallel using AVX512 instructions.

3. OPTIMIZATIONS

In Section 2 we described algorithmic improvements to `WHFast512`. In this section we focus on the implementation details that make this algorithm run fast.

3.1. *Assembly*

The first version of `WHFast512` was written in C99 with special intrinsics that map directly to AVX512 instructions. This approach makes it easy to include AVX512 in existing code. However, one disadvantage is that it does not allow fine-grained control over various aspects of the code because intrinsics leave those details to the compiler. For example, the compiler allocates registers and decides which variables to put on the stack.

For these reasons, the main parts of the new version of `WHFast512` are now written in about 800 lines of x86 assembly. We make extensive use of macros, which allow us to reuse much of the code for different configurations. Specifically, macros generate 24 slightly different functions for simulations with different number of planets, with and without general relativistic corrections, with and without encounter checks, as well as with and without ejection checks (see Section 2.2). Similarly, macros

Input: X, β, nTerms
Output: $Gs_{0,1,2,3}$
$z \leftarrow \beta X^2$
`/* Stumpff functions by Horner's method */`
$c_2 \leftarrow 1/(\text{nTerms} - 1)!$
$c_3 \leftarrow 1/\text{nTerms}!$
`/* Plain Horner for the higher order terms */`
for $k \leftarrow \text{nTerms} - 2, \text{nTerms} - 4, \ldots, 7$ **do**
  $c_3 \leftarrow 1/k! - z\, c_3$
  $c_2 \leftarrow 1/(k-1)! - z\, c_2$
`/* Compensated Horner for the last two terms */`
$d_2 \leftarrow 0$
$d_3 \leftarrow 0$
for $k \leftarrow 5, 3$ **do**
  $p \leftarrow z\, c_3$
  $p_e \leftarrow \text{fma}(z, c_3, -p)$
  $c_3 \leftarrow 1/k! - p$
  $s_e \leftarrow (1/k! - c_3) - p$
  $d_3 \leftarrow (s_e - \delta_k - p_e) - z\, d_3$
  $p \leftarrow z\, c_2$
  $p_e \leftarrow \text{fma}(z, c_2, -p)$
  $c_2 \leftarrow 1/(k-1)! - p$
  $s_e \leftarrow (1/(k-1)! - c_2) - p$
  $d_2 \leftarrow (s_e - \delta_{k-1} - p_e) - z\, d_2$
$c_2 \leftarrow c_2 + d_2$
$c_3 \leftarrow c_3 + d_3$
`/* Stiefel functions from Stumpff functions */`
$Gs_2 \leftarrow X^2\, c_2$
$Gs_3 \leftarrow X^3\, c_3$
$Gs_1 \leftarrow X - \beta\, G_3$
$Gs_0 \leftarrow 1 - \beta\, G_2$

Algorithm 2. Pseudocode of our Stiefel function evaluation. The Stumpff functions c_2 and c_3 are evaluated by Horner's method from their truncated Taylor series, and the Stiefel functions follow from the recurrence relations (S. Mikkola 1997). The last two Horner steps of each series use a compensated summation that accumulates the rounding of every operation in a low order word d, which is added back at the end. Here, $\text{fma}(a, b, c) = ab + c$ is evaluated with a single rounding, and δ_k is the precomputed rounding error of the coefficient $1/k!$. This compensation is applied only in the final evaluation that produces the f and g functions; intermediate evaluations use plain Horner for all terms.

create six different functions for symplectic correctors. This approach avoids branching because we can select the most appropriate kernel once at the beginning of the integration.

3.2. *Keeping the Simulation in Registers*

AVX512 supports 32 CPU registers, each 512 bits wide. Using only seven registers, we can keep the entire simulation state, specifically the masses, positions, and velocities of all particles, in registers at all times. The remaining registers are sufficient for performing all computations and storing intermediate values and constants such as the timestep and numerical factors.

We still need to use memory to store some constants, such as the 128 matrix coefficients (see below) as these alone would take up half the available registers. However, by keeping the simulation state in registers, we never have to write to memory during an integration except when an output is required. Thus, the cache is not polluted and can work very efficiently. In fact, we have zero cache misses[7]. As a result, `WHFast512`'s performance is not limited by the speed of memory access in contrast to many other HPC applications.

3.3. *Matrix Multiplications*

As discussed in Section 2.1, we need to perform several matrix multiplications to transform from Jacobi coordinates to heliocentric coordinates and back. In the case of eight planets, we need to multiply an 8×8 matrix by an 8-element vector containing x, y, or z coordinates. Thus, in total, we need to perform six matrix-vector multiplications per timestep. It is therefore crucial to optimize this calculation as much as possible. We achieve very good performance by combining several approaches.

First, the matrix elements are constants stored in memory. Because we never write to memory (see above), they remain in the CPU's L1 cache, and load operations are therefore very efficient.

Second, we perform the multiplications for three coordinates at the same time. This way, we need to load each matrix element only once and can reuse it for all three coordinates.

Third, we use two accumulators per coordinate. The accumulators are only combined at the end of the multiplication. This allows the CPU to schedule instructions in parallel and avoids data dependencies in the pipeline.

Fourth, we use fused multiply-add instructions extensively. These instructions improve both throughput and accuracy by avoiding an intermediate rounding step.

3.4. *Avoiding Branches*

Branches such as those originating from conditional statements or loops can significantly slow down a simulation. This is because modern CPUs often execute instructions out-of-order, but branches with multiple possible outcomes make such execution more difficult. We therefore carefully remove almost all branches in `WHFast512`'s kernel and are left with only two.

The first branch controls the overall loop over individual timesteps. This branch allows the loop to terminate

[7] Cache misses might still occur if the operating system performs some other operation on the CPU during an integration.

after the specified number of timesteps. The other remaining branch is in the Kepler step, where it checks for convergence (see Section 2.3).

3.5. *Square Roots*

The interaction step is dominated by the pairwise force evaluations between planets, each of which is proportional to $1/r^3$, where r is the distance between the two particles. No single instruction computes $1/r^3$, and on current CPUs and both the square root and the division instruction have a low throughput and high latency. We therefore use the approximate reciprocal square root instruction `rsqrt14`, which returns an estimate of $1/\sqrt{x}$ accurate to about 14 bits. The `rsqrt14` instruction has a latency of about 8 clock cycles compared to over 50 for the normal square root plus division instructions. We already have r^2 from the distance calculation, so applying `rsqrt14` to r^2 returns an estimate of $1/\sqrt{r^2} = 1/r$. Similar to A. Tanikawa et al. (2012), we refine this estimate with two Newton-Raphson iterations, and then obtain $1/r^3$ with two additional multiplications. Doing so makes the reciprocal square root accurate to machine precision. This procedure replaces the expensive square root and the division instructions with a short sequence of multiply and fused-multiply-add instructions, all of which are high throughput and low latency.

In comparison to the actual IEEE 754 compliant square root and division instructions, the result of our procedure is not guaranteed to be correctly rounded. However, in the case of the interaction step, the rounding is not crucial as a) the accelerations we calculate are small to begin with as they are perturbations to the Keplerian motion, b) because they get multiplied by a small number, the timstep, and c) because we encounter the same rounding error for both particles and thus, thanks to Newton's thirs law, energy is conserved regardless.

In the Kepler step we need both the distance r as well as its reciprocal $1/r$: r is used to compute ζ, and $1/r$ to compute β and the first order guess for X. Here the square root is evaluated only once per planet and timestep, and not once for each of the $N\cdot(N-1)/2$ pairs as in the interaction step. It therefore does not lie on the throughput-critical path, and the Kepler step remains dominated by the later iterative solver. Furthermore, because the result of the square root operation is used at the end of the Kepler step to advance the positions and velocities, any bias would propagate to those quantities. We thus avoid the `rsqrt14` instruction used for the pairwise forces and use the IEEE 754 compliant square root and division instructions in the Kepler step instead.

4. TESTS

4.1. *Kepler solver*

The Kepler solver of `WHFast512` is crucial for both the speed and accuracy of the entire integrator. We therefore test the Kepler solver extensively with one particle at a time. We use a fixed timestep of 5 days and plot the results over a grid of semi-major axis and eccentricity. The phase of the orbit is chosen randomly.

Figure 2 shows the relative semi-major axis error, the sign of the semi-major axis error, as well as the number of iterations in the Kepler solver. We also overplot the current orbital parameters of Mercury and Venus. The vertical dashed lines correspond to the planets' semi-major axes. The other dashed lines correspond to locations in parameter space where a particle crosses the orbit of Mercury or Venus. The solid white line shows where the pericenter timescale (J. Wisdom 2015) is equal to the timestep. These lines help to understand whether the accuracy in a region of parameter space matters or not. For example, if the timestep is larger than the pericenter timescale (top left above the solid white line), then we fail to resolve physically relevant timescales. Similarly, if a planet becomes very eccentric, then the planet is likely to have a close encounter with a neighbouring planet shortly afterwards. In both cases, the Kepler solver is no longer the limiting factor for the accuracy of a simulation.

The left panel of Fig. 2 shows that, in the most relevant region of parameter space, the Kepler solver is accurate to machine precision, $\sim 10^{-16}$. The middle panel shows that the sign of the error is random in the same region. This is important for having an unbiased integrator (see Section 2.3). For example, imagine we have an integrator accurate to 10^{-16} after one timestep but with a bias of the same magnitude (i.e., the sign of the error is the same every time). After, say, 10^{11} steps corresponding to a few Gyr in a Solar System integration, the error in the semi-major axis would have increased to 10^{-5} from this error source alone. If, on the other hand, the integrator is unbiased, then the error would only be $10^{-16} \cdot 10^{11/2} \approx 3 \cdot 10^{-11}$.

The right panel shows the number of iterations required for the Kepler solver to converge. Since we always have two Halley steps followed by at least one Newton step, the minimum number of iterations is 3. For a large region of parameter space, we indeed converge in only 3 iterations. More iterations are required for higher eccentricities and shorter orbital periods. The yellow colour indicates where we fail to converge in 7 iterations and thus have to fall back to the bisection method. This only occurs for very eccentric orbits. Note that even in

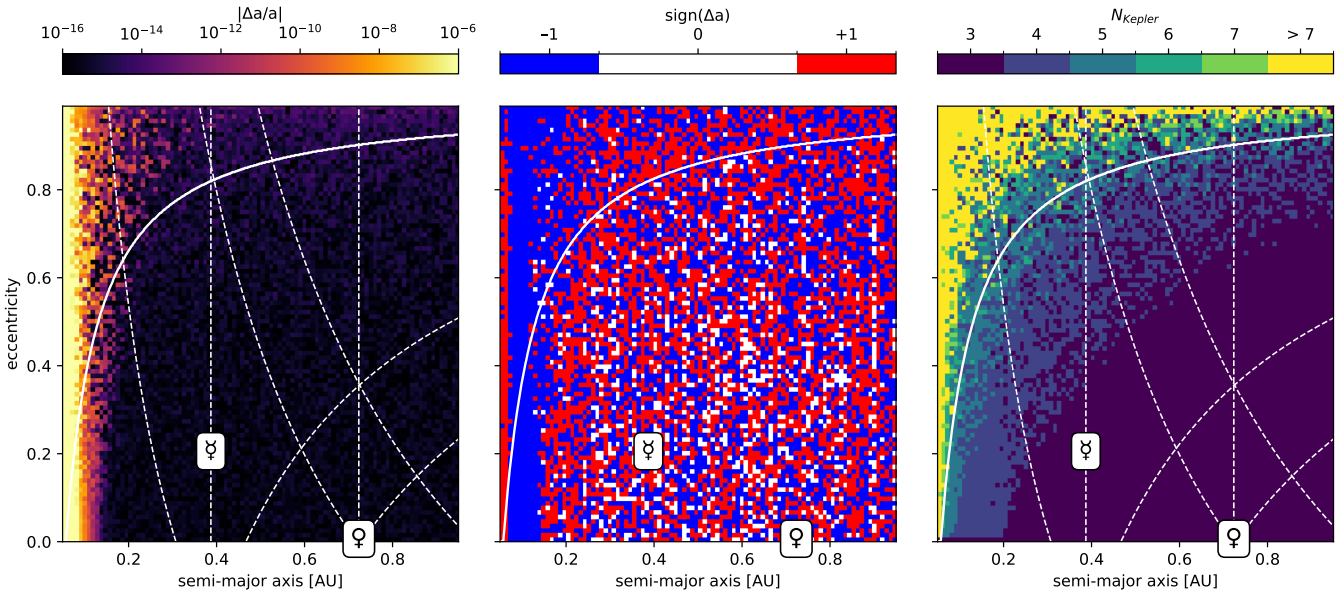

Figure 2. The absolute relative semi-major axis error, the sign of the semi-major axis error, and the number of iterations in the Kepler solver. Here, a test particle is integrated for one timestep of 5 days. The current locations of the Solar System planets are shown in each plot. See the text for a description of the lines.

the case where the bisection method was used, the error remains close to machine precision and the sign of the error remains random.

The Kepler solver fails for very short orbital periods. This is expected because our algorithm does not support timesteps that are larger than the orbital period. Fixing this is easy enough to do (it is done in `WHFast`), but would introduce an additional branch in the code which would affect the speed of the algorithm. Since simulations in which the timesteps are large compared to the innermost planet's period lead to unphysical results anyway (J. Wisdom 2015), we decided to not implement a Kepler solver which unconditionally converges in all cases.

Finally, we repeat the tests from Fig. 2 with the same 5 day timestep but now integrate each particle for 1000 orbital periods instead of only one timestep. The results are shown in Fig. 3. As expected, the relative semi-major axis error is now larger as errors accumulate. The sign of the semi-major axis error is still random in the bottom right region of parameter space, indicating that the Kepler solver remains unbiased here. However, the area in which the sign is consistently positive or negative has increased. This result reveals a biased contribution to the error. As this term grows $\propto t$, it will dominate over the unbiased $\propto \sqrt{t}$ term over sufficiently long timescales. We show in Section 4.3 that the timescale over which this bias becomes apparent in a simulation of the Solar System is longer than the age of the Solar System.

4.2. *Convergence*

We test the convergence of `WHFast512` by integrating the Solar System with all eight planets for 100 years. The relative energy error at the end of the integration is plotted as a function of the timestep in Fig. 4 for different integrators.

The new `WHFast512` integrator converges as the timestep gets smaller, as expected for a second order scheme. When symplectic correctors are turned on, the errors are reduced by a further factor of $\sim 10^3$ for typical timesteps ~ 5 days. For timesteps smaller than ~ 2 days, the error starts to increase again as the timestep is reduced further because we reach the floor imposed by double precision floating point arithmetic.

Note that the old version of `WHFast512` performs significantly worse because it uses democratic heliocentric coordinates rather than Jacobi coordinates and does not support symplectic correctors. For the same timestep, the new version of `WHFast512` is 5 orders of magnitude more accurate.

4.3. *Longterm Integrations*

We test the long-term conservation properties of `WHFast512` by simulating the Solar System with all eight planets and general relativistic corrections for 10 Gyr. We use a timestep of 6 days and turn on symplectic correctors. The initial conditions of each simulation are

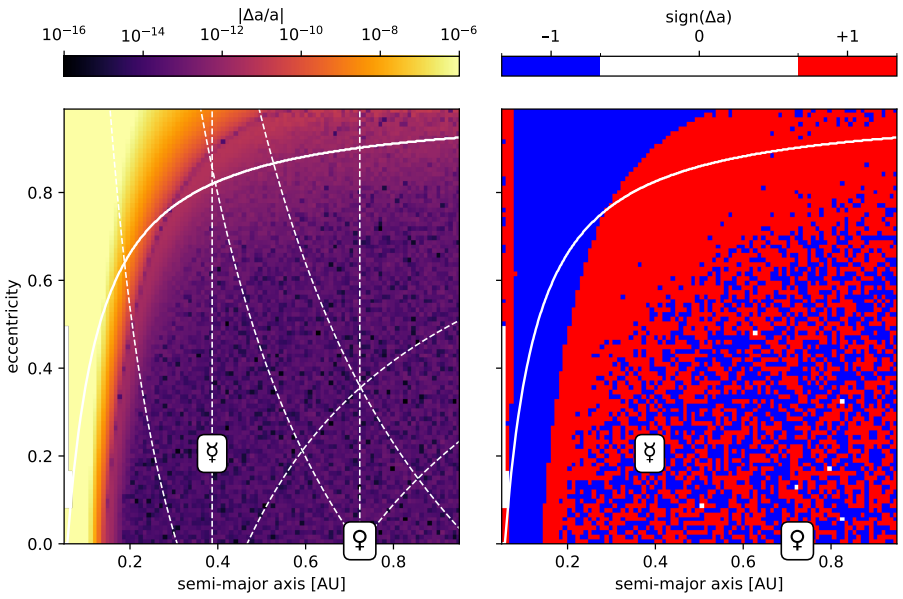

Figure 3. Same as the first two panels of Fig. 2, but each particle is integrated for 1000 orbital periods. The top and left of the plots show a biased error growth. The bottom right of the plots show an unbiased error growth in the most important area of parameter space.

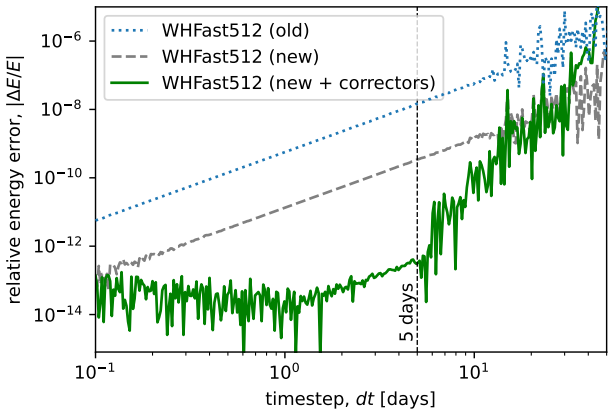

Figure 4. Relative energy error as a function of the timestep for different integrators in a 100 year simulation of the Solar System with all eight planets.

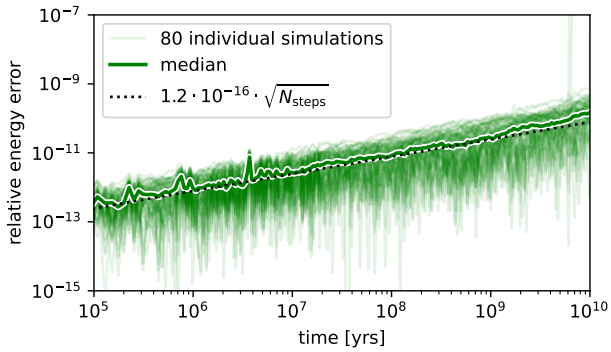

Figure 5. The relative energy error as a function of time in 80 simulations of the Solar System. The bold black line shows the median. The dotted line shows the expected behavior for an unbiased integrator that is accurate to machine precision after one timestep. Two simulations go unstable at late times, leading to a large energy error.

perturbed by a few meters in a random direction. The relative energy errors of the 80 simulations and their median are shown as a function of time in Fig. 5.

The median relative energy error remains below 10^{-10} over 10 Gyr. The errors grow as $\Delta E/E \sim 1.2 \cdot 10^{-16} \cdot \sqrt{N_{\rm steps}}$. This shows that the integrator is unbiased and follows Brouwer's law (D. Brouwer 1937) over physically relevant timescales. We expect that over much longer timescales, we will eventually see linear error growth (see the discussion in Section 4.1).

Note that two simulations go unstable at late times which leads to a spike in their energy errors. Instabilities in the Solar System are expected and the rate of instability is consistent with previous results (J. Laskar & M. Gastineau 2009; H. Rein et al. 2026).

4.4. *Runtime*

To measure the runtime, we compare `WHFast512` to the non-SIMD `WHFast`, one of the fastest N-body integrators for planetary systems (P. Javaheri et al. 2023). We once again use the Solar System with all 8 planets, a 6 day timestep, and general relativistic corrections as the canonical test case. All results presented here are

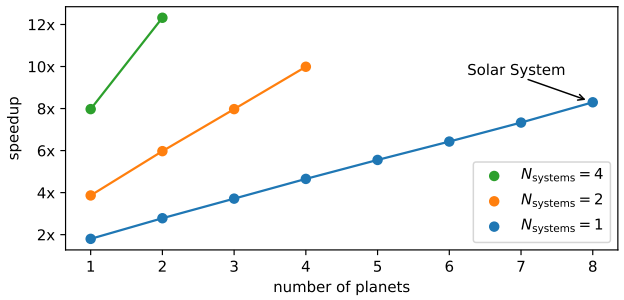

Figure 6. The speedup of `WHFast512` compared to `WHFast` in simulations with different number of planets. For `WHFast512` simulations that use $N_{\text{systems}} > 1$, the corresponding `WHFast` simulation are run sequentially.

from simulations performed on an AMD EPYC 9655 2.6GHz processor. Similar speedups were measured on an Intel Xeon Gold 6148 2.4GHz CPU.

We were able to finish a 5 Gyr integration in only 8.7 hours. This corresponds to a 60% speedup when compared to a simulation with the earlier version of `WHFast512` using the same timestep and CPU. When compared to the non-SIMD version of `WHFast`, this corresponds to a more than 8× speedup. For `WHFast`, aggressive compiler optimizations were used (`-O3`, `-march-native`). For `WHFast512` the specific compiler or compiler optimizations do not matter as it is written in assembly.

If the number of planets N is ≤ 4 `WHFast512` can integrate multiple systems at the same time as we can pack one AVX512 register with data from up to 8 planets and they don't necessarily need to be in the same system. The speedup for different numbers of planets and different numbers of planetary systems integrated at the same time are shown in Fig. 6. As one can see, the speedup is linear in the number of planets if the number of systems is kept fixed. `WHFast512` performs best when 8 planets are integrated at the same time as this fills up one AVX512 register. For the case of 2 planets in 4 different planetary systems, `WHFast512` is more than 12 times faster than `WHFast`.

4.5. *A Note on Units*

In the tests in this section, we use physical units to describe the setup and results. For example, we use a timestep of 5 days. Since gravity is scale invariant, this does not lead to a loss of generality. What matters physically is the timestep relative to the orbital period.

However, we introduce a scale into the problem by setting the Kepler solver's convergence criterion to a fixed value ϵ, which has units of time divided by length. In practice, this means that the units should be chosen such that positions and velocities are not excessively large or small. For example, having the innermost planet at 0.01 AU or 100 AU is perfectly fine, but having a planet at 10^{11}m or 10^{-12}Mpc is not. If necessary, ϵ can to be adjusted accordingly.

Furthermore, `WHFast512` is hardcoded to use units in which the gravitational constant is $G = 1$. This allows us to avoid multiplications by G entirely. We also hardcode the speed of light when general relativistic corrections are used. Thus, in practice and without loss of generality, the code units used by `WHFast512` measure length in astronomical units and time in yr/2π.

5. DISCUSSION

We argue that, with the improvements described in Sections 2 and 3, we approach the optimal performance attainable by a Wisdom-Holman type integrator on a standard CPU. There are several reasons why further significant improvements (speedups of $\gtrsim 2\times$) will be hard to achieve.

First, the code is entirely compute bound. Thus, the latency and throughput of instructions, particularly floating point operations, are the limiting factors for `WHFast512`. Since we do not use memory to store the simulation state and read only a small amount of memory for coordinate transformations, the code is never memory bound. This is in contrast to many other scientific HPC applications where memory access is the main bottleneck. As a result, higher memory bandwidth, unified memory, in-memory processing, and other memory related technologies are irrelevant to our application.

Second, if we optimize one part of the algorithm, we are limited by the fraction of the total runtime contributed by that part (Amdahl's argument, G. M. Amdahl 1967). In our case, the main parts of the algorithm are the interaction step, the Kepler step, and the matrix-vector multiplications used for coordinate transformations. These take up approximately 25%, 63%, and 11% of the total runtime, respectively.

Thus, the Kepler step is the most promising target for further optimization. Within the Kepler step, about 50% of the time is spent in the iterations solving Kepler's equation. As we have shown in Section 4.1, we require only three iterations to converge to a solution for most orbits. We must perform at least two iterations to check for convergence. Thus, we can remove at most one of the three iterations, for example by finding a better initial guess, using data from previous timesteps, or using a higher order solver. Even under the optimistic assumption that such a change would not involve any additional computational work, this would give us a speedup of at most $2/3 \cdot 50\% \cdot 63\% \approx 20\%$. Some improvements might be possible with a different formulation of the Kepler

problem itself, for example one that does not rely on the Stiefel functions or the f and g functions.

Next, we consider the interaction step. Here, the main bottlenecks are the throughput and latency of the square root and division instructions needed for the force calculations. We already use the faster, 14 bit accurate `rsqrt14` instruction with only two refinement steps. Thus, without hardware changes, such as providing more floating point units for each CPU core, we do not expect a significant speedup of the interaction step.

Finally, even a substantial improvement to the matrix-vector multiplication would yield an overall speedup of at most 11%.

For all these reasons, we believe that our implementation of `WHFast512` is almost optimal for simulations of planetary systems with $N \leq 8$ planets on the current generation of high-end CPUs.

6. CONCLUSIONS

In this paper, we have presented a new version of the `WHFast512` N-body integrator. This Wisdom-Holman style algorithm has been highly optimized for planetary systems with $N \leq 8$ planets. It is by far the fastest N-body integrator currently available for this kind of simulation. Using a timestep of $dt = 6$ days, `WHFast512` can integrate the eight planets of the Solar System for 5 Gyr in under 8.7 hours on an AMD EPYC 9655 processor. On the same hardware, this is 60% faster than the previous version of `WHFast512` and more than 8× faster than the non-SIMD version of `WHFast`, which is itself already significantly faster than most N-body codes (P. Javaheri et al. 2023).

We achieved this speedup by switching from C to assembly, giving us much finer control over individual registers and CPU instructions. Specifically, this rewrite allowed us to keep the entire simulation state in CPU registers, reduce the number of branches in the code to just two per timestep, and implement a highly optimized matrix-vector multiplication for the coordinate transformations.

In addition to a significant speedup, we also switched `WHFast512` to Jacobi coordinates, allowing us to use symplectic correctors and thus suppress errors by up to 5 orders of magnitude. We took great care to make our implementation of the Kepler solver as bias-free as possible, thereby enabling reliable long-term integrations.

Although our discussion has focused on the Solar System as the canonical test case, `WHFast512` is a reliable and fast N-body integrator for many small-N simulations. It is particularly well suited for simulations that determine the stability of planetary systems because these simulations are typically stopped as soon as orbits cross or close encounters occur.

We argue that `WHFast512` is almost optimal in terms of performance and accuracy for these kind of simulations on present day CPUs. Small improvements might still be possible, but the two main bottlenecks are now the square root calculations in the interaction step and the finite number of iterations ($\sim$3) in the Kepler solver. On the hardware side, the only changes that would improve the runtime are an increase in clock speed or an increase in floating point execution units. Neither of those changes seem likely given the current trend towards high parallelism and low precision aimed at machine learning applications.

Future work might focus on applying similar techniques to those presented in this paper to larger, arbitrary-N simulations and to non-symplectic or hybrid-symplectic integrators that are capable of accurately resolving close encounters.

`WHFast512` is freely available in the `REBOUND` integrator package at https://rebound.hanno-rein.de. `REBOUND` is compiled with `WHFast512` by default on 64-bit x86 systems. At runtime, `REBOUND` checks for the relevant CPU flags and reports an error if the CPU does not support AVX512 instructions. `WHFast512` is also available in precompiled python wheels for Linux and Windows.

ACKNOWLEDGMENTS

We thank Daniel Tamayo and Dang Pham for helpful discussions and Dang Pham for help with testing `WHFast512` on Windows. This research has been supported by the Natural Sciences and Engineering Research Council (NSERC) Discovery Grant RGPIN-2020-04513. The authors did not use generative AI for code generation, or writing and editing this paper.

REFERENCES

Abbot, D. S., Hernandez, D. M., Hadden, S., et al. 2023, The Astrophysical Journal, 944, 190

Amdahl, G. M. 1967, in Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, AFIPS '67 (Spring) (New York, NY, USA: Association for Computing Machinery), 483–485, doi: 10.1145/1465482.1465560

Applegate, J. H., Douglas, M. R., Gursel, Y., Sussman, G. J., & Wisdom, J. 1986, Astronomical Journal, 92, 176, doi: 10.1086/114149

Batygin, K., & Laughlin, G. 2008, The Astrophysical Journal, 683, 1207

Beust, H. 2003, A&A, 400, 1129, doi: 10.1051/0004-6361:20030065

Boué, G., Laskar, J., & Farago, F. 2012, Astronomy and Astrophysics, 548, A43, doi: 10.1051/0004-6361/201219991

Brouwer, D. 1937, Astronomical Journal, 46, 149, doi: 10.1086/105423

Brown, G., & Rein, H. 2023, MNRAS, 521, 4349, doi: 10.1093/mnras/stad719

Cohen, C. J., & Hubbard, E. C. 1965, AJ, 70, 10, doi: 10.1086/109674

Danowitz, A., Kelley, K., Mao, J., Stevenson, J. P., & Horowitz, M. 2012, Communications of the ACM, 55, 55

Eckert, W. J., Brouwer, D., & Clemence, G. M. 1951, Astronomical papers prepared for the use of the American ephemeris and nautical almanac ; v. 12, 12, 1

Esmaeilzadeh, H., Blem, E., St. Amant, R., Burger, D., & Barker, K. 2011, IEEE Micro, 32, 28

Farrés, A., Laskar, J., Blanes, S., et al. 2013, Celestial Mechanics and Dynamical Astronomy, 116, 141, doi: 10.1007/s10569-013-9479-6

Javaheri, P., Rein, H., & Tamayo, D. 2023, The Open Journal of Astrophysics, 6, 29, doi: 10.21105/astro.2307.05683

Kinoshita, H., & Nakai, H. 1984, Celestial Mechanics, 34, 203, doi: 10.1007/BF01235802

Kinoshita, H., Yoshida, H., & Nakai, H. 1991, Celestial Mechanics and Dynamical Astronomy, 50, 59

Laskar, J. 2008, Icarus, 196, 1, doi: 10.1016/j.icarus.2008.02.017

Laskar, J., & Gastineau, M. 2009, Nature, 459, 817, doi: 10.1038/nature08096

Mikkola, S. 1997, Celestial Mechanics and Dynamical Astronomy, 67, 145, doi: 10.1023/A:1008217427749

Moore, G. E. 1965, Electronics, 38, 114

Moore, G. E. 1975, in Technical Digest of the 1975 International Electron Devices Meeting (IEDM), Vol. 21 (IEEE), 11–13

Newhall, X. X., Standish, E. M., & Williams, J. G. 1983, A&A, 125, 150

Nitadori, K., Makino, J., & Hut, P. 2006, NewA, 12, 169, doi: 10.1016/j.newast.2006.07.007

Nobili, A., & Roxburgh, I. W. 1986, in IAU Symposium, Vol. 114, Relativity in Celestial Mechanics and Astrometry. High Precision Dynamical Theories and Observational Verifications, ed. J. Kovalevsky & V. A. Brumberg, 105–110

Quinn, T. R., Tremaine, S., & Duncan, M. 1991, AJ, 101, 2287, doi: 10.1086/115850

Rein, H., Dey, K., & Tamayo, D. 2026, arXiv e-prints, arXiv:2601.08019, doi: 10.48550/arXiv.2601.08019

Rein, H., & Tamayo, D. 2015, MNRAS, 452, 376

Rein, H., & Tamayo, D. 2019, Research Notes of the AAS, 3, 16, doi: 10.3847/2515-5172/aaff63

Rein, H., Tamayo, D., & Brown, G. 2019, Monthly Notices of the Royal Astronomical Society, 489, 4632, doi: 10.1093/mnras/stz2503

Saha, P., Stadel, J., & Tremaine, S. 1997, Astronomical Journal, 114, 409, doi: 10.1086/118485

Stiefel, E. L., & Scheifele, G. 1975, Linear and regular celestial mechanics. Perturbed two-body motion. Numerical methods. Canonical theory.

Tanikawa, A., Yoshikawa, K., Okamoto, T., & Nitadori, K. 2012, NewA, 17, 82, doi: 10.1016/j.newast.2011.07.001

Wisdom, J. 2015, AJ, 150, 127, doi: 10.1088/0004-6256/150/4/127

Wisdom, J., & Holman, M. 1991, Astronomical Journal, 102, 1528, doi: 10.1086/115978

Wisdom, J., Holman, M., & Touma, J. 1996, Fields Institute Communications, 10, 217

Wisdom, J. L. 1981, PhD thesis, California Institute of Technology, Pasadena.